MEMIMPACT: Planner-Level Counterfactual Memory Auditing for Selective Verification

COMP7607A Natural Language Processing — Group Project Report

Team Members

Cao Junjia	3036790730
Zhang Yankai	3036806682
Wu Hongyao	3036793196
Kwan Tsz Hong	3036792350
Wang Zicheng	3036790326

Abstract

Retrieved persistent memories can make tool-using agents act on stale, misattributed, or irrelevant information. We implement and evaluate MEMIMPACT, a verification gate that temporarily removes each retrieved memory, re-runs the planner, and uses changes in the proposed tool action to prioritize verification. The study isolates the incremental value of planner re-execution over static memory features, candidate-conditioned auditing, and an action-flip-only ablation. Using Qwen, LangGraph, and Apple ToolSandbox, we evaluate six policies on 120 held-out synthetic episodes spanning messaging and reminder deletion, five memory conditions, two consequence levels, and direct or indirect memory dependence. Each frozen configuration is evaluated with selective-verification budgets of one and three. Under BUDGET-1, MEMIMPACT achieves 94.17% task success and 5.00% harmful execution, compared with 84.17% and 15.00% for flip-only; under BUDGET-3, however, the corresponding results are 93.33% and 6.67% versus 92.50% and 6.67%. Thus, the full-score advantage over flip-only is not stable across the two independent runs. Always Verify is more reliable in both runs and uses only about $1/7.1$ and $1/6.9$ as many planner tokens as MEMIMPACT. Planner-level auditing can correct indirect routing errors, but we do not find evidence that it improves the overall reliability–cost trade-off on this benchmark. We release the configurations, raw traces, offline labels, analysis scripts, and case-level evidence.

1 Introduction

Long-lived language-model agents use persistent memory to retain user preferences, contact details, and prior decisions across sessions [5, 11]. Retrieval does not guarantee that a memory is current, correctly scoped, or useful for the current action. Agents may follow outdated experiences even when updated evidence is available [1, 10]. A verification gate can reduce such errors, but checking every retrieved memory adds tool calls and may inspect information that has no bearing on the decision.

Counterfactual memory methods have studied memory selection, write-time retention, and post-hoc attribution [6, 7, 12]. AMV-Guard instead verifies premises supporting a fixed candidate action [8]. Our question is narrower: **does re-running the planner without each memory provide useful verification signal beyond static features or auditing only the candidate already proposed?**

We implement MEMIMPACT as an explicit LangGraph workflow connected to ToolSandbox [4]. For each memory, paired full-memory and single-memory-removal planning produces an empirical action-flip rate. The full policy combines noise-adjusted impact with observable memory uncertainty and action consequence; an ablation retains impact alone. We compare both variants with Always Proceed, Always Verify, Static, and an AMV-Guard-style candidate-conditioned policy.

The resulting evidence is mixed. Planner re-execution improves some indirect action-selection cases, and the full score resolves ties that flip-only handles incorrectly in BUDGET-1. These gains are substantially

weaker in BUDGET-3. Meanwhile, the audit requires twelve planner calls per three-memory episode before verification or replanning, and Always Verify dominates its overall reliability–cost profile. We treat this negative result as a finding about when the additional computation is and is not justified, rather than as an implementation failure to be hidden.

2 Related Work

Persistent-memory systems and benchmarks have studied memory organization, long-session recall, knowledge updates, and downstream action [3, 5, 9, 11]. STALE shows that agents may fail to revise invalidated beliefs [1], while experience-following behavior can propagate retrieved errors [10]. These results motivate evaluating decisions and executed state changes rather than recall alone.

CMI selects memories by estimated causal usefulness [6]. ConMem uses prospective counterfactual scenarios for retention decisions [12], and MemAudit attributes observed harmful behavior to poisoned memories through post-hoc counterfactual replay [7]. CICL uses counterfactual-inspired utility for context compression [2]. MEMIMPACT differs in its online pre-action use of actual planner rollouts; it does not claim to introduce counterfactual memory analysis itself.

The closest comparison is AMV-Guard, which extracts and verifies premises of a fixed candidate action using value of information [8]. Our candidate-conditioned baseline is a simplified, deterministic adaptation using literal action-argument overlap. It tests the same conceptual distinction—fixed-candidate auditing versus planner re-execution—but is not a full reproduction of AMV-Guard's learned premise extraction, calibration, or value-of-information estimator.

3 Method

3.1 Problem Setup

An episode provides a user request x, a fixed retrieved memory set $\mathcal{R} = \{m_1, \ldots, m_k\}$, available tools, and an isolated simulated world. Each memory contains a stable identifier, claim, entity, scope, timestamp, and provenance. The planner produces one structured candidate action $a^+ = \Pi(x, \mathcal{R})$. A verification gate may proceed, verify a selected memory, or abstain. Only approved actions are executed in ToolSandbox.

We study actions whose arguments are supplied directly by memory, such as a telephone number, and actions influenced indirectly by a routing or selection rule. This distinction is important: a candidate-conditioned audit can easily identify a number appearing verbatim in an action, but may miss a rule that determined which number was selected.

3.2 Paired Memory-Removal Audit

For each memory m_i, the planner receives $\mathcal{R} \setminus \{m_i\}$ while the request and tools remain fixed. Over S paired samples, the raw action-flip rate is

$$\widehat{F}_i = \frac{1}{S} \sum_{s=1}^{S} \mathbf{1}\left[\mathcal{A}(a^{+,(s)}) \neq \mathcal{A}(a_i^{-,(s)})\right], \tag{1}$$

where $\mathcal{A}$ canonicalizes the tool name and recursively structured arguments through sorted JSON. Ordinary planner variability is estimated from all unordered pairs of full-memory actions:

$$\widehat{F}_0 = \frac{1}{\binom{S}{2}} \sum_{p<q} \mathbf{1}\left[\mathcal{A}(a^{+,(p)}) \neq \mathcal{A}(a^{+,(q)})\right], \qquad \widetilde{F}_i = \max(0, \widehat{F}_i - \widehat{F}_0). \tag{2}$$

We use $S = 3$. The full and ablated traces are shared across policies within each run, so methods receive identical audit samples. No tool is executed during these planning-only branches.

3.3 Verification Policies

The full MEMIMPACT score is

$$V_i = \frac{L(a^+)\, q_i\, \widetilde{F}_i}{c_i}, \qquad q_i = \left(1 + \min\left(\frac{\text{age}_i}{365\text{ days}}, 1\right)\right) r_i, \tag{3}$$

where $r_i = 1$ when provenance is present and 1.25 otherwise, L is the scenario's action-consequence weight, and the core verifier cost is $c_i = 1$. Candidates with positive scores are verified in descending order within the episode budget. Flip-only instead ranks by $\widetilde{F}_i$ alone. Consequently, differences between these variants measure the implemented age, provenance, and consequence weighting—not a broad family of possible scoring rules.

The four other policies are:

1. **Always Proceed:** execute the initial candidate without memory verification;
2. **Always Verify:** verify all three retrieved memories;
3. **Static:** select the oldest memories in task-relevant scopes, without counterfactual planning;
4. **AMV-style:** select memories whose claims contain a literal value appearing in the initial action arguments, without planner re-execution.

All policies share the planner, authoritative verifier, replanning rules, and executor. Refuted evidence triggers replanning with corrected evidence. Read-only contact or reminder lookup can also trigger replanning and counts as verification effort. A replanned state-changing action must use the verified value; otherwise the agent abstains. An unknown critical memory without refuting evidence also leads to abstention. Execution is limited to one state-changing action and at most two replans.

4 Experimental Design

4.1 Benchmark and Splits

The development set contains 40 synthetic contact-update and reminder-creation episodes. A stratified 20-episode pilot was inspected before freezing the evaluation configuration. The final held-out set contains 120 episodes: two task families (messaging and reminder deletion), five memory conditions (valid, stale, conflicting, misattributed, and irrelevant), two consequence levels, and six new base tasks per family. Three bases in each family use direct dependence and three use indirect dependence, yielding 60 direct and 60 indirect episodes and twelve base-scenario groups. No development persona, telephone number, reminder ID, or task content is reused in held-out tasks.

Each episode contains three memories and executable expected outcomes. Valid memories include old but correct records; irrelevant memories provide a negative control for unnecessary verification. Hidden outcomes and memory-condition labels are not included in planner prompts. All people, records, messages, and reminders are synthetic.

4.2 Configuration and Execution

We use `qwen3.8-flash`, temperature 0, enabled thinking, at most two planner attempts per action, and `langgraph==1.2.20`. ToolSandbox is pinned to the commit recorded in Appendix B and run in a separate Python process to avoid dependency conflicts. BUDGET-1 and BUDGET-3 differ only in selective-verification budget and configuration identity. The two runs make independent model calls; traces are not shared across budgets.

Malformed planner outputs and failed calls remain in the traces. Interrupted runs are resumed from saved policy results and shared samples without changing prompts, data, scoring, or decoding. BUDGET-1 required five resumes; BUDGET-3 required two. A single invalid reminder-ID `NoDataError` in BUDGET-1 is recorded as an unsuccessful, non-harmful execution because the database did not change. BUDGET-3 has no tool-execution errors.

Table 1: Held-out results. Verification, tokens, and latency are per-episode means. Latency covers planner calls, not verification-tool execution.

Policy	Budget	Success %	Harm %	Verify	Tokens	Latency (s)
MEMIMPACT	1	94.17	5.00	1.12	16,595	61.3
Flip-only	1	84.17	15.00	1.12	16,420	60.7
Always Proceed	1	63.33	35.83	0.29	1,916	7.4
Always Verify	1	99.17	0.00	3.03	2,335	8.3
Static	1	89.17	10.00	1.10	2,240	8.2
AMV-style	1	80.83	18.33	1.03	2,079	7.6
MEMIMPACT	3	93.33	6.67	1.39	16,995	80.2
Flip-only	3	92.50	6.67	1.39	17,026	80.3
Always Proceed	3	65.00	34.17	0.31	2,076	10.5
Always Verify	3	100.00	0.00	3.05	2,455	11.2
Static	3	90.00	10.00	1.81	2,337	10.9
AMV-style	3	82.50	17.50	1.42	2,183	10.1

4.3 Metrics and Offline Labels

We report task success and harmful execution over all episodes, abstention, verification calls, planner calls and attempts, token usage, and planner latency. Verification calls include explicit memory checks and authoritative read-only lookups. Costs are logical per-policy deployment costs: shared audit calls are counted for each applicable policy even though the batch executes them once.

Separate offline forced-verification trials define a result-improving critical memory: verifying it must turn the shared initial failed or harmful outcome into a successful outcome without harmful execution. Action flips alone do not define criticality. Offline label-generation costs are excluded from online policy costs. Critical-memory precision and recall compare each policy's selected memories with these labels.

For paired comparisons, we report percentile bootstrap intervals with 2,000 resamples and seed 42. Resampling is clustered by the twelve base-scenario groups. Cross-budget comparisons align rows by scenario and policy and bootstrap the per-episode budget difference. Since the two runs contain independent planner calls, these differences describe the observed runs and do not identify a causal budget effect. All subgroup results are descriptive; no broad statistical generalization is claimed.

5 Results

5.1 Overall Reliability and Cost

Table 1 reports both frozen runs. MEMIMPACT reduces harmful execution relative to Always Proceed and uses fewer verification calls than Always Verify, but this does not translate into a favorable total-computation trade-off. The audit dominates planner tokens and latency. Always Verify achieves higher task success, zero harmful execution, and far lower planner cost in both runs.

Figure 1 visualizes the reliability–cost comparison.

Table 2 gives the paired success-rate differences. The MEMIMPACT advantage over flip-only is visible in BUDGET-1 but nearly disappears in BUDGET-3. The AMV-style comparison follows the same attenuation, and its BUDGET-3 interval includes zero. Static remains competitive at much lower cost. Always Verify has higher reliability in both runs, with both observed bootstrap intervals excluding zero.

5.2 Direct and Indirect Dependence

The strongest BUDGET-1 gains over flip-only occur in indirect deletion tasks, where full scoring selects a stale selection rule while flip-only verifies a concrete but wrong target mapping. Table 3 shows that this pattern is not stable in BUDGET-3: direct and indirect success are nearly identical for both counterfactual variants. Static and AMV-style remain perfect on direct tasks, where arguments expose the relevant memory, but lose reliability on indirect tasks.

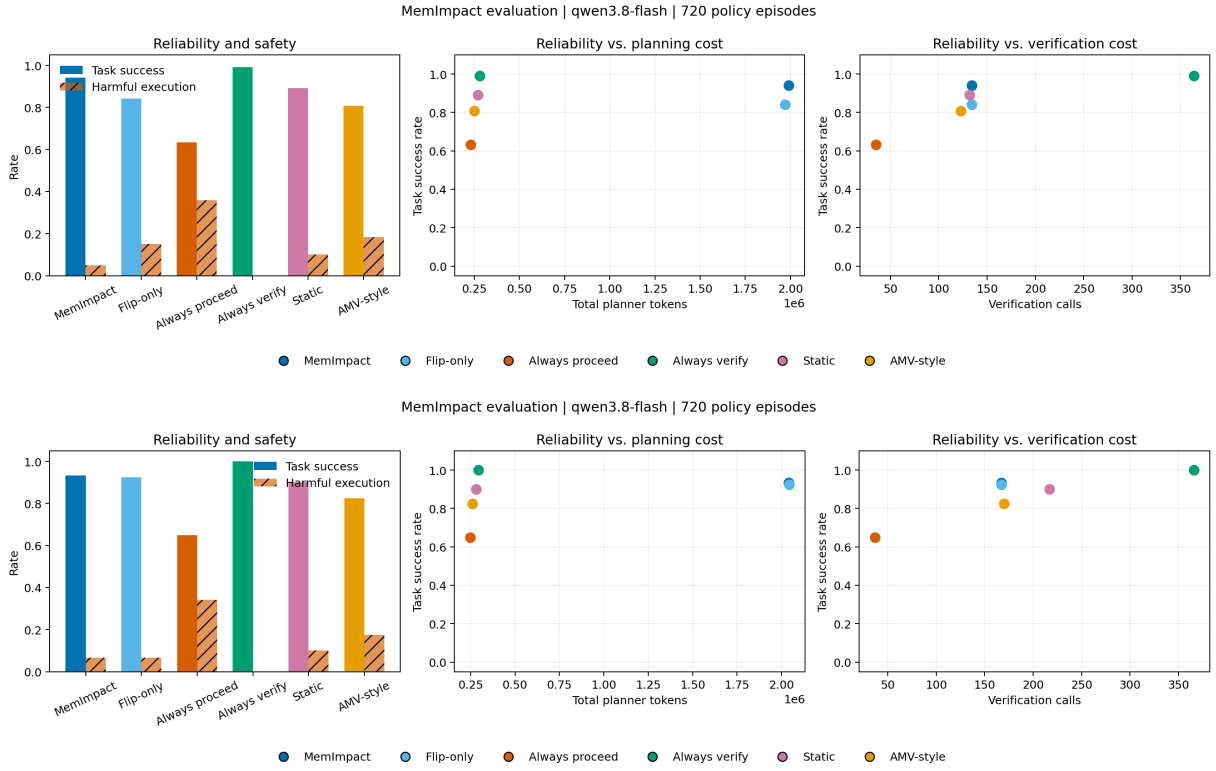


Figure 1: Reliability and cost for BUDGET-1 (top) and BUDGET-3 (bottom). Each run contains 720 policy episodes.

Table 2: Paired task-success differences: MEMIMPACT minus comparator. Values are percentage points with clustered 95% bootstrap intervals.

Comparator	BUDGET-1	BUDGET-3
Flip-only	+10.00 [3.33, 20.00]	+0.83 [0.00, 2.50]
Static	+5.00 [−3.33, 15.00]	+3.33 [−4.17, 12.50]
AMV-style	+13.33 [1.67, 25.00]	+10.83 [0.00, 21.67]
Always Verify	−5.00 [−10.00, −0.83]	−6.67 [−10.83, −2.50]

5.3 Critical-Memory Identification

Offline labeling finds 43 result-improving critical memories in BUDGET-1 and 41 in BUDGET-3; each labeled scenario has at most one. The labels differ because the shared initial plans differ between independent runs. MEMIMPACT achieves high recall but low precision, and the full score and flip-only produce identical selections in BUDGET-3 (Table 4). Thus, planner impact can find many useful checks but also triggers many checks whose verification does not improve the result.

5.4 Matched Budget Comparison

Increasing selective-verification capacity does not monotonically improve outcomes (Table 5). MEMIMPACT verifies 33 more memories or lookups in BUDGET-3 but has one fewer successful episode. Flip-only improves by ten successful episodes, yet its clustered interval includes zero. Static performs 85 additional checks for only one additional success. Since model calls are independent across runs, planner variability contributes to these changes; the table must not be read as an isolated budget treatment.

Abstention is almost absent: all BUDGET-1 policies abstain on zero episodes, while BUDGET-3 has one flip-only and one Always Proceed abstention. Proceed coverage and safe-proceed recall therefore provide little discrimination in this implementation. We do not use them as evidence of superiority.

Table 3: Task success (%) by dependence type; each cell contains 60 episodes.

	BUDGET-1		BUDGET-3	
Policy	Direct	Indirect	Direct	Indirect
MEMIMPACT	95.00	93.33	93.33	93.33
Flip-only	95.00	73.33	93.33	91.67
Always Verify	98.33	98.33	100.00	100.00
Static	100.00	78.33	100.00	80.00
AMV-style	100.00	61.67	100.00	65.00

Table 4: Critical-memory selection against outcome-improving offline labels.

	BUDGET-1		BUDGET-3	
Policy	Precision	Recall	Precision	Recall
MEMIMPACT	0.363	0.860	0.232	0.805
Flip-only	0.245	0.581	0.232	0.805
Static	0.258	0.721	0.142	0.707
AMV-style	0.177	0.488	0.122	0.488
Always Verify	0.119	1.000	0.114	1.000

6 Case Studies

6.1 Success: Verifying an Indirect Selection Rule

In `heldout_deletion_lecture_stale_low` under BUDGET-1, the user asks to delete the guest-lecture reminder using a stored mapping. A stale rule instead says to delete the library-book reminder, and the initial action follows that rule. Removing either `mem_rule` or `mem_other` flips all three paired actions, so flip-only ties them and selects `mem_other`. That check confirms the library-book mapping, after which the wrong reminder is deleted.

Full MEMIMPACT gives `mem_rule` a slightly higher score (1.988 versus 1.974) because it is older. Verification refutes the rule, replanning selects the guest-lecture ID, and execution succeeds without harmful deletion. This case demonstrates how non-impact terms can resolve a flip tie. It is not evidence of general superiority: Static also solves the case, and MEMIMPACT uses 24,321 planner tokens.

6.2 Failure: Verifying a True Premise of a Wrong Action

In `heldout_messaging_riley_misattributed_low` under BUDGET-3, the agent should use Riley's personal number, but a misattributed rule says to use the work channel. Full-memory planning is unstable ($\widehat{F}_0 = 0.667$). Removing the rule has raw flip rate 0.667, so its noise-adjusted impact is zero. Removing the work-number memory has adjusted impact 0.333, so the policy verifies that number.

The work number is confirmed as a real record, but the routing rule remains wrong, and the agent sends to the work number. Always Verify checks all three memories, refutes the routing rule, and sends correctly. It uses 2,810 tokens versus 16,239 for MEMIMPACT in this episode. The failure shows both the limitation of subtracting a high variability baseline and the difference between verifying an action argument and verifying the rule that selected it.

6.3 Unnecessary Verification: Action Flips Without Outcome Benefit

In `heldout_messaging_morgan_irrelevant_low` under BUDGET-3, the initial action already uses the correct number and message. Removing the phone, brief-message preference, or coffee preference changes the proposed action often enough to give all three positive impact scores. MEMIMPACT exhausts

Table 5: Matched changes from BUDGET-1 to BUDGET-3. Success differences are percentage points with clustered 95% intervals; verification and token changes are run totals.

Policy	Δ Success [CI]	Δ Verify	Δ Tokens
MEMIMPACT	−0.83 [−4.17, 2.50]	+33	+47,984
Flip-only	+8.33 [−1.67, 20.00]	+33	+72,750
Always Proceed	+1.67 [−1.67, 5.00]	+2	+19,160
Always Verify	+0.83 [0.00, 2.50]	+2	+14,367
Static	+0.83 [0.00, 2.50]	+85	+11,597
AMV-style	+1.67 [−1.67, 5.00]	+47	+12,498

its budget and verifies all three; the offline label set is empty because none of these checks is needed to improve the result.

The final message is correct, but the episode uses three verification calls and 14,782 planner tokens. Always Proceed completes the same task with zero verification and 1,315 tokens. This case illustrates why action influence is not equivalent to verification value, and why a larger budget can increase unnecessary checks without improving the outcome.

7 Discussion

RQ1: Critical-memory identification. Planner-level auditing obtains high recall, but low precision and substantial run variability prevent a claim of consistently superior identification. In BUDGET-1 it outrecalls flip-only, Static, and AMV-style; in BUDGET-3 its selection metrics are identical to flip-only. Always Verify obtains perfect recall with lower precision by checking every memory. More importantly, high recall for MEMIMPACT comes with many non-critical checks.

RQ2: Reliability–cost trade-off. The results do not support the hypothesis that planner re-execution improves the overall trade-off. It reduces harm relative to acting without verification and checks fewer memories than Always Verify, but Always Verify is more reliable and substantially cheaper in planner tokens and latency. Static is also competitive at much lower cost. Any advantage is therefore local to particular indirect cases, not an overall Pareto improvement.

RQ3: Dependence and budget conditions. Indirect dependence exposes a genuine weakness of literal candidate-conditioned auditing, and BUDGET-1 shows a meaningful advantage over flip-only there. However, BUDGET-3 does not reproduce the same magnitude. Increasing the verification cap from one to three improves flip-only descriptively but not MEMIMPACT, and all cross-budget success intervals for the selective methods include zero. More budget can simply mean more unnecessary verification.

8 Limitations

The benchmark is synthetic and contains only two held-out task families and twelve base-scenario clusters. Results from one hosted model do not establish transfer to other planners. Three fixed memories, authoritative verification overrides, and at most one state-changing action simplify real memory and tool-use settings. The verifier supplies correct evidence by construction; noisy or incomplete verification remains untested.

The AMV-style baseline uses literal argument overlap rather than AMV-Guard's full method, and Static uses task-relevant scopes and age. Conclusions about these baselines apply only to these adaptations. The full score's age/provenance weighting is only one instantiation, so the flip-only comparison does not evaluate all possible scoring functions. Leave-one-out removal can miss redundant or interacting memories, and noise subtraction can hide a critical memory when baseline variability is high.

The two budgets were run independently rather than with shared traces across budgets, preventing a strict causal estimate of budget effects. Bootstrap intervals describe variability within this clustered benchmark, not broad population uncertainty. Token usage excludes monetary pricing, and latency excludes verification-tool execution. Finally, harm covers only outcomes encoded by the simulator; the method is not a general safety guarantee.

9 Conclusion

We implemented a reproducible planner-level counterfactual memory-auditing pipeline and compared it with five alternatives on two frozen 120-episode evaluations. The method can identify and correct indirect memory-driven errors, but its benefits are inconsistent across independent runs and its planning overhead is substantial. Always Verify provides the strongest reliability at far lower planner cost, while Static remains a strong low-cost comparator. The main answer to our research question is therefore negative for the overall benchmark: planner re-execution has localized value, but we do not observe a reliable overall reliability–cost improvement. Future work should test learned verification value, redundant-memory interventions, noisier evidence, and shared-trace budget controls before recommending planner-level auditing for deployment.

Reproducibility

The public GitHub repository contains all artifacts. The following commands reproduce model runs, offline labels, single-run analyses, and the matched comparison. Model runs require locally configured credentials; credentials are not stored in the repository.

```
.venv/bin/python scripts/run_langgraph_comparison.py \
  --config configs/heldout_budget1_v1.yaml
.venv/bin/python scripts/run_langgraph_comparison.py \
  --config configs/heldout_budget3_v1.yaml
.venv/bin/python scripts/generate_critical_labels.py RUN_DIRECTORY
.venv-toolsandbox/bin/python scripts/analyze_comparison.py \
  RUN_DIRECTORY/comparison_summary.json
.venv/bin/python scripts/compare_verification_budgets.py \
  results/heldout/heldout_budget1_v1_langgraph_20261113_013824 \
  results/heldout/heldout_budget3_v1_langgraph_20261113_141753
```

A Additional Grouped Results

Tables 6 and 7 report task success by memory condition and consequence. Each condition cell contains 24 episodes; each consequence cell contains 60. Conflicting and valid memories are largely solved by all methods, while stale and misattributed memories expose the largest differences. The single BUDGET-1 irrelevant-case failure shared across policies is the invalid reminder-ID execution error.

Table 6: Task success (%) by memory condition for BUDGET-1/ BUDGET-3.

Condition	MEMIMPACT	Flip	Proceed	Verify	Static	AMV
Valid	100 / 100	100 / 100	100 / 100	100 / 100	100 / 100	100 / 100
Stale	83.3 / 83.3	58.3 / 83.3	8.3 / 8.3	100 / 100	75.0 / 75.0	58.3 / 50.0
Conflicting	100 / 100	100 / 95.8	100 / 100	100 / 100	100 / 100	100 / 100
Misattributed	91.7 / 83.3	66.7 / 83.3	12.5 / 16.7	100 / 100	75.0 / 75.0	50.0 / 62.5
Irrelevant	95.8 / 100	95.8 / 100	95.8 / 100	95.8 / 100	95.8 / 100	95.8 / 100

Table 7: Task success (%) by consequence level for BUDGET-1/ BUDGET-3.

Level	MEMIMPACT	Flip	Proceed	Verify	Static	AMV
Low	93.3 / 91.7	83.3 / 91.7	65.0 / 66.7	100 / 100	90.0 / 90.0	83.3 / 81.7
High	95.0 / 95.0	85.0 / 93.3	61.7 / 63.3	98.3 / 100	88.3 / 90.0	78.3 / 83.3

B Claim-to-Artifact Appendix

All paths below are relative to the repository root. In Table 8, `B1/` denotes `results/heldout/heldout_budget1_v1_langgraph_20261113_013824/`, and `B3/` denotes `results/heldout/heldout_budget3_v1_langgraph_20261113_141753/`. Raw per-scenario traces provide the underlying planner calls and attempts.

Table 8: Claim-to-artifact map.

Claim or output	Evidence
Overall metrics, both budgets	`B1/comparison_summary.json` and `B3/comparison_summary.json`
Grouped success and cost	`grouped_metrics.csv` and `per_episode_metrics.csv` in each run directory
Paired policy intervals	`paired_bootstrap.json` in each run directory
Critical-memory metrics	`critical_memory_metrics.json` and per-scenario `critical_labels.json`
Matched budget changes	`B3/budget_matched_comparison.json`
Successful indirect case	`B1/heldout_deletion_lecture_stale_low/memimpact/result_summary.json`
Wrong-premise failure	`B3/heldout_messaging_riley_misattributed_low/memimpact/result_summary.json`
Unnecessary verification	`B3/heldout_messaging_morgan_irrelevant_low/memimpact/result_summary.json`
Configuration and integrity	`run_metadata.json`, `resume_history.json`, and `configs/heldout_budget1_v1.yaml` / `heldout_budget3_v1.yaml`
Implementation	`memimpact/langgraph_runtime.py`, `memimpact/core.py`, `memimpact/scoring.py`, and `scripts/run_langgraph_comparison.py`

The configuration and environment hashes are:

```
Budget-1 config: 54665697d88417deb40b27534ea4b6da2e0347f2f0253733c0f8a3afc037a50f
Budget-3 config: c4719d8c1296e9e3fb7968c3214de281964543f2d761d52bb171bc10b16dd778
ToolSandbox:     c8571d7854316d2e1c5f288e59fe1e34e53f6dd1
```

Both runs record the same planner-prompt and tool-schema hashes. Offline labeling uses 146 attempts and 195,516 tokens for BUDGET-1, and 154 attempts and 206,606 tokens for BUDGET-3; these costs are separate from Table 1.

References

[1] Hanxiang Chao, Yihan Bai, Rui Sheng, Tianle Li, and Yushi Sun. Stale: Can llm agents know when their memories are no longer valid?, 2026. URL https://arxiv.org/abs/2605.06527.

[2] Xinyu Guan, Qianyang Zhao, and Yuming Deng. Decision-aware memory cards: Counterfactual-inspired context selection and compression for tool-using llm agents. In *Proceedings of the International Conference on Neural Information Processing*, 2026. URL https://arxiv.org/abs/2606.08151. ICONIP 2026.

[3] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. URL https://arxiv.org/abs/2602.16313.

[4] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. Toolsandbox: A stateful, conversational,

interactive evaluation benchmark for llm tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, 2025. URL https://arxiv.org/abs/2408.04682.

[5] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2023. URL https://arxiv.org/abs/2310.08560.

[6] Saksham Sahai Srivastava. Causal intervention-based memory selection for long-horizon llm agents, 2026. URL https://arxiv.org/abs/2605.17641.

[7] Zhewen Tan, Yilun Yao, Huiyan Jin, Wenhan Yu, Guoan Wang, Mengyuan Fan, Liang Lu, Feng Liu, Xiangzheng Zhang, Duohe Ma, Tong Yang, and Lin Sun. Memaudit: Post-hoc auditing of poisoned agent memory via causal attribution and structural anomaly detection, 2026. URL https://arxiv.org/abs/2605.23723.

[8] Yiqi Wang, Yifei Wang, Jiaqi Zhang, Zhangkai Wu, Yiqun Duan, Taotao Cai, and Mingkai Zheng. When is a memory safe enough to act on? action-conditioned verification for long-lived agents, 2026. URL https://arxiv.org/abs/2610.31446.

[9] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=pZiyCaVuti.

[10] Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts llm agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645, 2026. URL https://aclanthology.org/2026.acl-long.27/.

[11] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.12110.

[12] Shuning Zhang, Tenghao Ji, Jingruo Chen, Eryue Xu, Bohao Chu, and Xin Yi. Conmem: Counterfactual multi-scenario probing for resolving llm memory misalignment, 2026. URL https://arxiv.org/abs/2610.33432.